Appendix B: The Harness Engineer: Review Less, Ship More

Introduction

Three weeks ago we approved a pull request we hadn't read. Not skimmed — hadn't opened. And it was the correct decision.

The PR had passed a 60-case eval suite. The guardrail layer reported zero violations. An adversarial reviewer agent — whose only job is to attack, and whose critique history exceeds the reliability of manual review — had grilled the diff and signed off. A second agent had verified the docs against the running code. Every one of those checks was something we built, and every one of them is stricter than human manual review.

Out of curiosity we pulled our review stats for the quarter. We're reading about a tenth of the code we read a year ago. Output is up so much we've stopped trusting our own measurements. The explanation isn't that we got smarter or that the models did — it's that our job quietly changed underneath us. We don't review lines anymore. We build the thing that reviews lines. Most of our week goes into harnesses: the specs, tests, evals, docs, and guardrails that get set up *before* any agent writes a word.

That's the shift this chapter is about. Chapters 10 through 20 built the machinery. This one is about what you do with your hands once the machinery works.

The Inversion

Here's the uncomfortable arithmetic. An agent fleet can produce more code in a day than you can read in a week. If your quality strategy is "a human reviews everything," your throughput is capped at the speed of manual review — and the fleet idles while you scroll. You've automated the writing and kept the bottleneck.

The way out is not reviewing faster. It's moving your effort one level up:

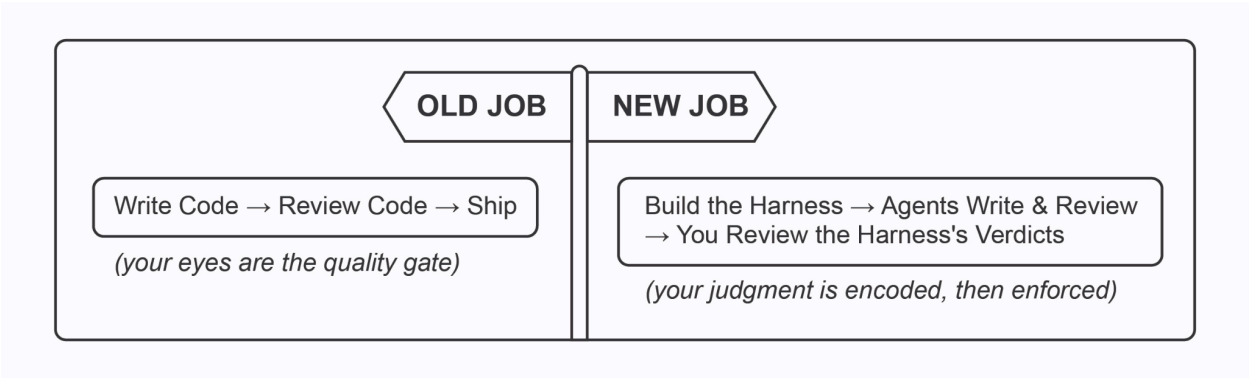


Figure 1: The Inversion: Moving from manual code review to encoded quality harnesses.

Everything the harness sets up, we've already built somewhere in this book. The spec is the `AGENTS.md` contract from Chapter 10. The tests are the coverage floor from §10.8. The evals are Chapter 18. The guardrails are Chapter 14. The loop that runs them without us is Chapter 20. What changes now is the *proportion*: those artifacts stop being the supporting cast around our coding time and become the work itself. When we say we spend our week on harnesses, we mean it literally — the ratio of harness time to code time in our calendars has inverted, and the code got better, not worse.

One thing we want to say plainly, because the simplified industry perspectives of this pitch skips it: reviewing 10x less code is only safe *because* of the ten chapters of infrastructure underneath it. Skip the eval suite and the guardrails and the escalation logic, and "reviewing less" has an older, insufficient validation. The harness earns us the right to look away. We don't get to claim the right without paying for it.

The Adversarial Fleet

Once the harness exists, you stop deploying agents one at a time and start deploying them in opposition. Chapter 6 introduced the debate protocol — Generator drafts, Challenger attacks — as a pattern for high-stakes answers. Scale that instinct up and it becomes an organizational principle: for every agent doing work, there's another one whose job is to break it.

A working fleet, in practice, splits along these lines:

- **Builders** write the code, the documentation, the deck for Thursday's review — the artifacts.
- **Adversaries** challenge assumptions and stress-test what the builders made. They don't suggest improvements; they hunt for the input that breaks it, the requirement it silently dropped, the assumption you baked into the spec that was wrong on day one.
- **Operators** run the evals continuously, watch the infrastructure, and execute the experiments — the Chapter 16 daemons and Chapter 20 loops, now numerous.
- **Improvers** reason over the results and feed fixes back into the harness and the guardrails themselves — the self-improving layer from §20.3, pointed at everything.

The practical advice hiding in that structure: **review the disagreements, not the output.** When the builder and the adversary agree, the work is almost always fine — that's what our eval history will show after a month. When they disagree, that disagreement is the most information-dense artifact our system produces, and it deserves human eyes far more than a green diff does. Our reading queue used to pull requests. Now it's an argument.

And this is why the adversary can't be a politeness layer. LLMs are trained to please you; an agent asked to "review" will find three minor nitpicks and approve. The `/grill-me` stance from §20.4 — attack the plan, hunt for the flaw, no credit for agreeing — is what turns a second model call into an actual second opinion. If your adversary has never once blocked a builder, you don't have an adversary. You have an ineffective validation mechanism that bills by the token.

/learn — The Command That Compounds

Antigravity ships a fourth loop command alongside `/plan`, `/grill-me`, and `/goal`, and it's the one that changes the economics: **`/learn`**.

Here's the failure it kills. You spend 45 minutes steering an agent through a workflow — the quarterly report, say. You correct the format twice, remind it about the compliance footnote, fix the way it merges the regional data. By the end, the output is exactly right. Then next quarter you open a fresh session and do the whole dance again, because everything you taught it lived in a context window that no longer exists. I call it the Groundhog Day loop: paying the same training cost every time, forever, for an assistant with no yesterday.

`/learn` breaks the loop by extracting the win while it's still in context:

This code snippet demonstrates how to use the /learn command to teach an agent a new, repeatable workflow. It shows the agent extracting formatting requirements, data-merging steps, and style constraints from a task to create a new skill file. This process allows for the automation of complex, repetitive tasks by turning successful execution patterns into reusable skills.

```
None
@Agent /learn Review the execution steps we just took to complete
the

Q3 financial summary.

ABSTRACTION: Extract the formatting requirements, the data-merging
steps, and the style constraints into a repeatable workflow.
```

```
RULE GENERATION: Save it as an Agent Skill named generate-q-report.md
in the /skills folder.

DOCUMENT INGESTION: Scan the attached finance-guidelines.pdf and make every future run of this skill enforce our compliance standards.
```

The agent analyzes what it just did right, abstracts the specifics into a procedure, and writes it down as a skill file. Next quarter, nobody re-explains anything. You type the skill's name and the entire 45-minute lesson replays as a single command.

Three things compound here. The knowledge compounds — every completed task can leave the system permanently smarter about *your* workspace, your formats, your rules. The friction collapses — a paragraph of careful prompting becomes one invocation, which means it actually gets used instead of being reconstructed slightly differently each time. And the consistency locks in — the procedure lives in a file now, so it stops varying with how you happened to phrase things on a given morning.

If this sounds familiar, it should. `/learn` is the experience distillation from §20.3 with a steering wheel attached: instead of the harness deciding autonomously which trajectories to keep, you point at a success and say *that one*. And the artifact it produces is exactly the `SKILL.md` format from Chapter 9 — frontmatter, procedure, constraints — which means everything Chapter 9 taught you applies, including the security posture. A skill written by an agent gets the same pull-request review as a skill written by a colleague, because it *is* executable instructions. `/learn` and then merge blindly, and you've automated the injection of unreviewed behavior into every future session.

The Claude Code equivalent

There's no `/learn` command in Claude Code, and after you've internalized Chapter 9 you'll see why it barely needs one — the agent can already write files, and skills are just files:

This code snippet illustrates how to achieve the same result as the /learn command using Claude Code's file writing capabilities. It guides the agent to review previous steps, extract a repeatable procedure, and save it as a skill file for future use. This method allows you to distill successful procedures into reusable skills without needing a specialized command.

```
None
Review the steps we just took to produce the Q3 summary.

Extract the repeatable procedure — formatting rules, data-merge steps,

compliance constraints from finance-guidelines.pdf — and write it as a

skill at .claude/skills/generate-q-report/SKILL.md, with a description

that matches how I'd naturally ask for this task.

Do not include values from this quarter's data. Procedure only.
```

Next session, the skill shows up in the catalog and activates like any other. For lighter-weight lessons — a preference, a convention, a "never do X again" — the same move at a smaller scale is asking Claude Code to append a rule to `CLAUDE.md`, which loads into every future session. Either way, the last instruction in that prompt is the one that matters: distill the *procedure*, not the *data*. The first time we did this we got a beautiful skill with September's revenue figures fossilized inside it, and every future run confidently reported a quarter that had already ended.

Taking It Outside the Codebase

Everything in this chapter so far appears to be software-specific. It isn't. The pattern — encode your judgment in a harness, let agents do the volume, verify mechanically, distill the wins — works anywhere the work has a checkable structure. The trick people miss is that far more work has a checkable structure than they think.

Watch what happens when you write a `/goal` for a marketing task with the full Chapter 20 discipline — verifier, metrics, rollback:

This code snippet provides an example of using the /goal command for a marketing task with strict verification parameters. It demonstrates how to define a clear objective, specify verifiers for brand voice and content limits, and set up automatic rollback conditions for non-compliant output. By using this approach, you can apply rigorous, machine-enforceable standards to creative work, ensuring consistency and quality.

```
None
@MarketingAgent /goal Design a Q3 go-to-market ad matrix for our

enterprise SaaS tier. Jobs-To-Be-Done framing: Fortune 500

decision-makers only, messaging restricted to the top 3 ROI pain
points.

VERIFIER: Check every piece of copy against the uploaded
brand-voice

guidelines and the LinkedIn/Google ad character limits.

METRICS TO PASS: Flesch-Kincaid grade < 8. "I/we"-to-"you" ratio

below 0.3. Zero passive voice.

ROLLBACK: If copy exceeds a platform limit or uses a banned
buzzword

("synergy", "seamless", "paradigm"), scrap the draft and
regenerate.
```

Look at those metrics. A reading grade is a formula. A pronoun ratio is arithmetic. Passive voice and buzzwords are regex. Character limits are `len()`. Every single one is machine-checkable — which means this "creative" task has a real termination condition, which means it can run in a Chapter 20 loop, which means the agent grades its own homework and regenerates before you ever see the draft. The subjective "looks good to me" review is gone, replaced by numbers a shell script can verify. (And yes, that buzzword blocklist is the same trick this book's own editing pass uses. Harnesses all the way down.)

Once you see the move — *judgment goes in the harness, volume goes to the agents* — the domains line up:

Legal. The harness encodes the constitutional constraints, the controlling case law, the firm's risk positions — the things a partner would never delegate. The agents then draft the contracts, simulate opposing counsel attacking every clause, and hunt for the loopholes before the other side's associates do. The adversarial fleet, in a jurisdiction.

Finance. The harness is the risk limits and the regulatory rules, encoded as hard constraints rather than system-prompt suggestions. Inside those walls, agents stress-test portfolios, run continuous market experiments, and optimize yield — and the rollback protocol isn't cosmetic there, it's the compliance story.

Education. The harness holds the curriculum standards and the pedagogical lines you won't cross. The agents build hyper-personalized lessons, probe each student's actual misconceptions instead of the average student's, and evolve the material between sessions — `/learn`, applied to teaching itself.

Health and science. The harness sets the clinical and safety bounds — non-negotiable, human-owned, audited. The agents run the million simulated biochemistry experiments no grad student has the hours for and reason over the patterns in the results. Karpathy's 700-experiment loop from Chapter 23, pointed at a wet-lab pipeline instead of a training script.

What You Absolutely Need to Review

Reviewing less is not reviewing nothing, and knowing where your eyes still belong is the actual skill. Four places, in priority order:

1. **The specs and the goals.** The `AGENTS.md`, the termination conditions, the metrics that define "pass." These are where your judgment gets encoded, and an error here is multiplied by every loop iteration that follows. No other reading you do pays off like this.
2. **The eval diffs.** Not the eval results — the *changes* to the suite. A weakened threshold or a deleted red-team case is how a system rots invisibly while every dashboard stays green.
3. **The escalations.** When a Chapter 20 loop gives up and pages you, that's the system telling you it hit something outside its encoded judgment. Escalations are your backlog of harness improvements, if you read them that way.
4. **The self-modifications.** Every prompt the critic rewrote, every tool the harness synthesized, every skill a `/learn` produced. Anything that changes future behavior gets human review, full stop — that rule survives from Chapter 9 no matter how good the fleet gets.

Notice what's not on the list: the code, most days. That's the inversion completing itself. Your judgment didn't leave the system. It moved to where it scales.

Key Takeaways

- **Your eyeballs are the bottleneck.** A fleet writes faster than you can read. Move your effort from reviewing the work to building the harness that reviews the work — specs, tests, evals, guardrails, loops.

- **Review the disagreements, not the output.** When builder and adversary agents agree, ship it. When they disagree, that's the most information-dense artifact your system produces — spend your attention there.
- **An adversary that never blocks anything is a rubber stamp.** The `/grill-me` stance — attack, no credit for agreeing — is what makes a second model call an actual second opinion.
- **`/learn` turns wins into permanent skills** — experience distillation with a steering wheel. But an agent-written `SKILL.md` is executable behavior and gets human review before it's trusted, same as Chapter 9 demanded.
- **More work is machine-checkable than you think.** Reading grades, pronoun ratios, character limits, buzzword blocklists — encode the judgment, and "creative" tasks get termination conditions too. The pattern travels to legal, finance, education, and science; the certification burden travels with it.

Conclusion

The engineers we know who've made this shift describe the same feeling: a mild identity crisis, followed by relief. The identity crisis is real — if you measured your worth in lines read and written, both numbers just fell off a cliff. The relief comes when you notice what replaced them. Every hour into the harness pays out on every task the fleet runs, forever. Every distilled skill makes tomorrow's session start further ahead. Every adversary you sharpen catches a class of mistake permanently. The old job was linear. This one compounds.

Stop managing tasks. Build the thing that manages tasks, and spend your judgment where a machine can't spend it for you.

That's the last pattern this book has to teach. Every part is now on the bench — the loop, the harness, the fleet, the skills that compound. Nothing left to do but bolt it all together and see if the machine runs.